



Topic: Abstraction in Java – Full Detailed Notes

Abstraction in Java – Full Detailed Notes

What is Abstraction?

Definition (English + Marathi):

Abstraction म्हणजे एक process ज्यात आपण user ला काय **function** आहे हे दाखवतो पण तो **function internally** कसा काम करतो ते **hide** करतो.

"User केवळ feature वापरतो, त्यामागचा logic त्याला माहीत नसतो."

Real-Life Analogy:

Imagine तुम्ही ATM वापरताय. तुम्ही amount enter करता → Card swipe करता → पैसा मिळतो. But internally:

- Database कनेक्ट होतं
- Balance verify होतं
- Logs maintain होतात

पण हे सगळं तुमच्यासमोर **visible** नसतं.

तुम्हाला काय करायचं आहे तेच दिसतं - **how it happens is hidden. That's Abstraction.**

Why Abstraction?

- User ला logic न सांगता functionality वापरू देता
 - Code readability आणि reusability वाढते
 - Interface वापरून **3rd party API securely expose** करता येते
 - Large scale project मध्ये architecture maintain करायला मदत होते
-

Types of Abstraction in Java

Abstraction achieve करता येतो दोन मार्गांनी:

Abstract Class

Interface

1. Abstract Class – आधीच partially developed template

Syntax:

```
abstract class Vehicle {  
    abstract void start(); // no body  
    void stop() {  
        System.out.println("Vehicle stopped"); // concrete method  
    }  
}
```

Explanation:

- abstract keyword वापरून declare करावी लागते.
- यात abstract methods + concrete methods असू शकतात.
- Object directly create करता येत नाही.
- Subclass must override abstract methods.

Example:

```
abstract class Bike {  
    abstract void run(); // abstract method  
  
    void fuelType() {  
        System.out.println("Petrol");  
    }  
}  
  
class Honda extends Bike {  
    void run() {  
        System.out.println("Honda is running safely");  
    }  
}
```

Usage:

```
Honda h = new Honda();  
h.run();           // Output: Honda is running safely  
h.fuelType();      // Output: Petrol
```

Abstract Class FAQs:

Q. Abstract class मध्ये constructor का असतो?

Subclass जेव्हा super() call करतं, तेव्हा abstract class चा constructor invoke होतो.

Q. Concrete methods allowed आहेत का?

Yes, तुम्ही logic सह method define करू शकता.

2. Interface – Fully Abstract Blueprint

Syntax:

```
interface PaymentGateway {  
    void pay(int amount); // No body – public abstract by default  
}
```

Explanation:

- सर्व methods public abstract असतात by default
- Interface ला instantiate करता येत नाही
- Variables are public static final → constants
- Multiple inheritance support करतं

Example:

```
interface PaymentGateway {  
    void pay(int amount);  
}  
  
class GooglePay implements PaymentGateway {  
    public void pay(int amount) {
```

```
        System.out.println("Paid ₹" + amount + " using GPay");
    }
}
```

Usage:

```
PaymentGateway pg = new GooglePay();
pg.pay(1000); // Output: Paid ₹1000 using GPay
```

Why Interface is Important?

Let's say **Flipkart** wants to integrate **ICICI bank's payment system**.

ICICI wants to protect:

- Their database
- Logic
- Security

So, instead of giving full code, they just give an **interface**:

```
interface BankTransaction {
    void withdrawAmt(int amt);
}
```

Flipkart will use it like this:

```
BankTransaction b = new ICICIBank(); // class from ICICI
b.withdrawAmt(500);
```

Flipkart ला फक्त method signature दिसतो, logic नाही - and that's what abstraction achieves.

Interface FAQs:**Q. Interface मध्ये method ला body देऊ शकतो का?**

Java 8 पासून **default** आणि **static methods** ला body देता येतो.

Q. Interface मध्ये constructor का नसतो?

कारण object तयार होत नाही.

Q. Interface extends/implements कैसे होते?**Type**

Interface extends Interface

Class implements Interface

Class extends Class implements Interface

Syntax

interface C extends A, B {}

class X implements Y {}

class X extends Y implements Z {}

Difference Between Abstract Class & Interface

Feature	Abstract Class	Interface
Constructor	Present	Not Allowed
Method Types	Abstract + Concrete	Only Abstract (till Java 7)
Variables	Any Type	Always public static final
Multiple Inheritance	No	Yes
Usage	Extend	Implement
Access Modifiers	Can be any	Only public

TECHNORBIT Special Examples:**✓ Abstract Class Example:**

```
abstract class Account {  
    abstract void openAccount();  
  
    void showBalance() {  
        System.out.println("Balance shown");  
    }  
}
```

✓ Interface Example:

```
interface Transaction {  
    void deposit(int amount);  
}  
  
class SBI implements Transaction {  
    public void deposit(int amount) {  
        System.out.println("Deposited ₹" + amount + " in SBI");  
    }  
}
```

TECHNORBIT TIP:

Interface वापरा जेव्हा तुम्हाला full abstraction पाहिजे

Abstract class वापरा जेव्हा काही methods ready आहेत आणि काही subclass complete करणार

